

Smart Plant Watering System with Arduino Uno

1. Introduction

1.1. Project Description

The goal of this project is to build a smart plant watering system that uses current soil moisture levels to automatically water a potted plant. The soil's moisture content is determined by the system using a soil moisture sensor. The mechanism turns on a water pump to irrigate the plant if the soil is dry and also shows real-time information on an LCD screen.

1.2. Objectives

- To ensure that the plant receive adequate watering even when user is busy or away.
- To achieve a sustainable and efficient water usage.

2. Design and Schematics

2.1. Materials Used

1. Arduino Uno
2. Soil Moisture Sensor
3. Small Water Pump (5V)
4. Relay Module (to control the water pump)
5. Water Tubing (for directing water to the plant)
6. Water Container (for holding the water supply)
7. Breadboard and Jumper Wires
8. Power Supply (USB cable for Arduino)
9. LCD Display or LEDs (for visual indicators)
10. Plant and Pot
11. Push button

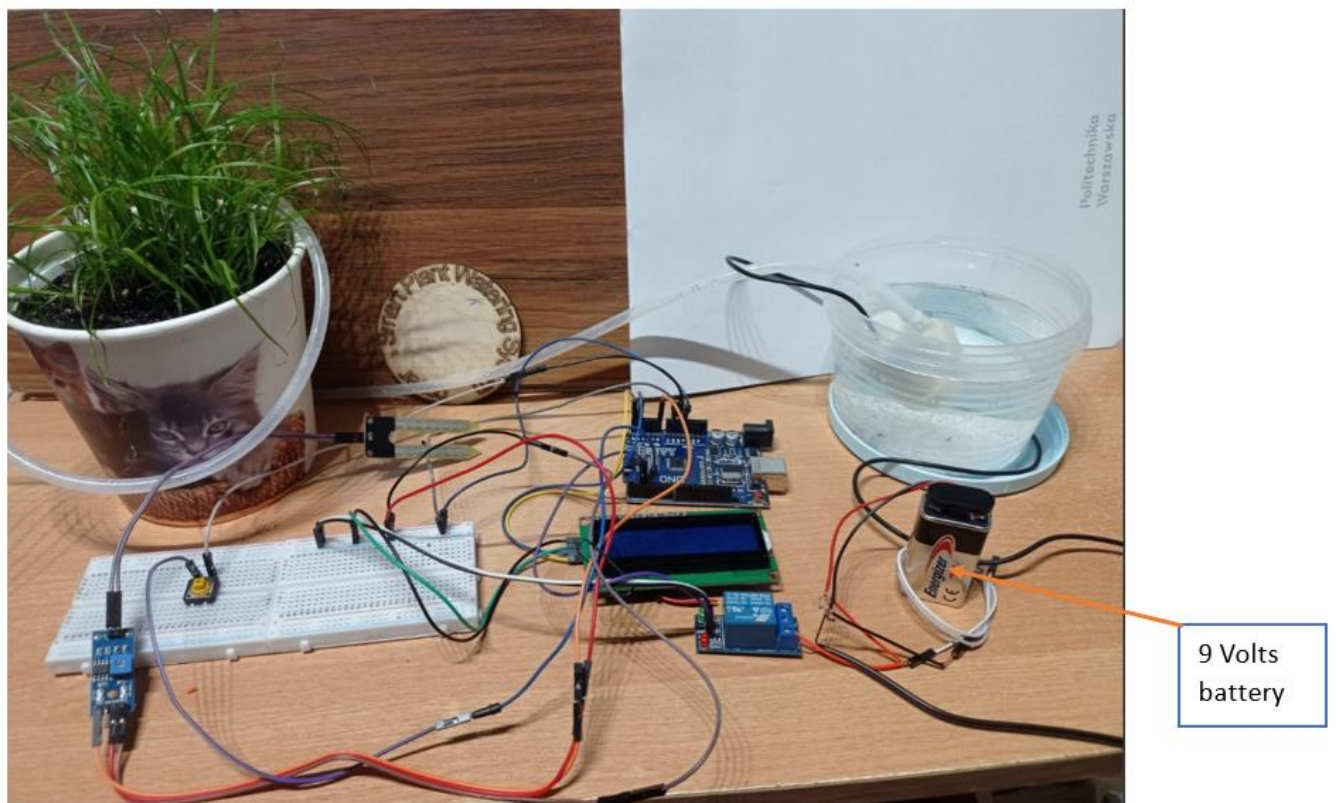


Figure 1. System Setup

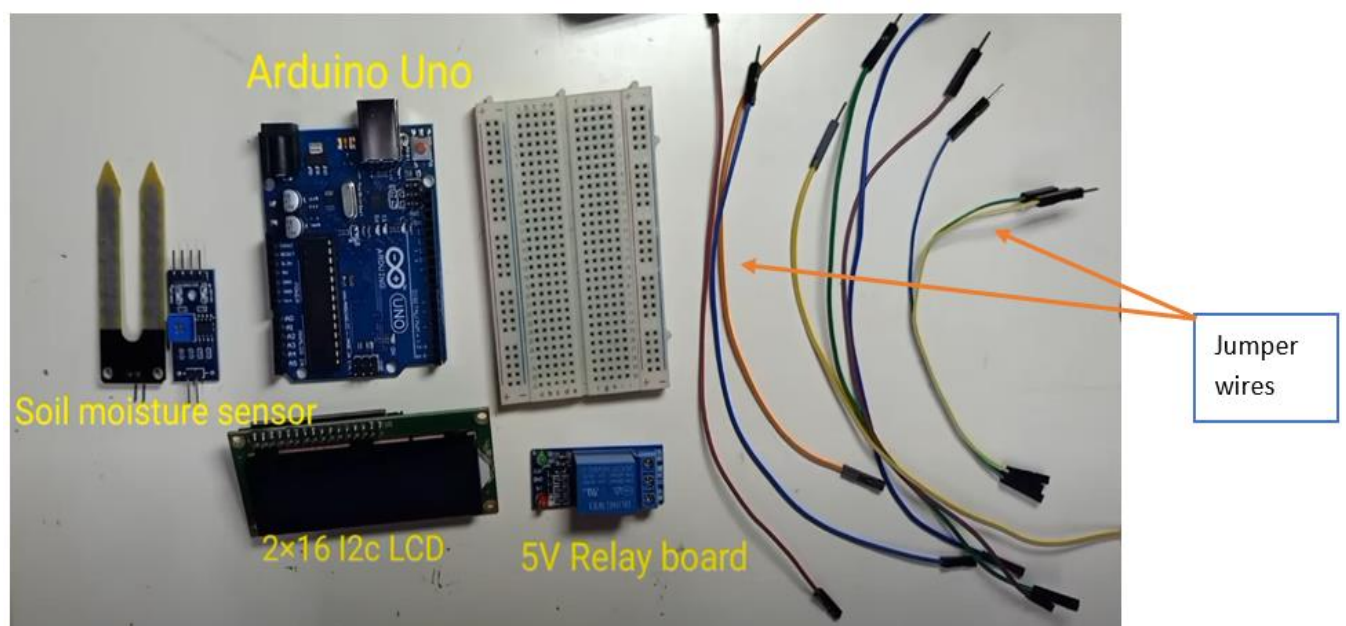


Figure 2. Materials and components

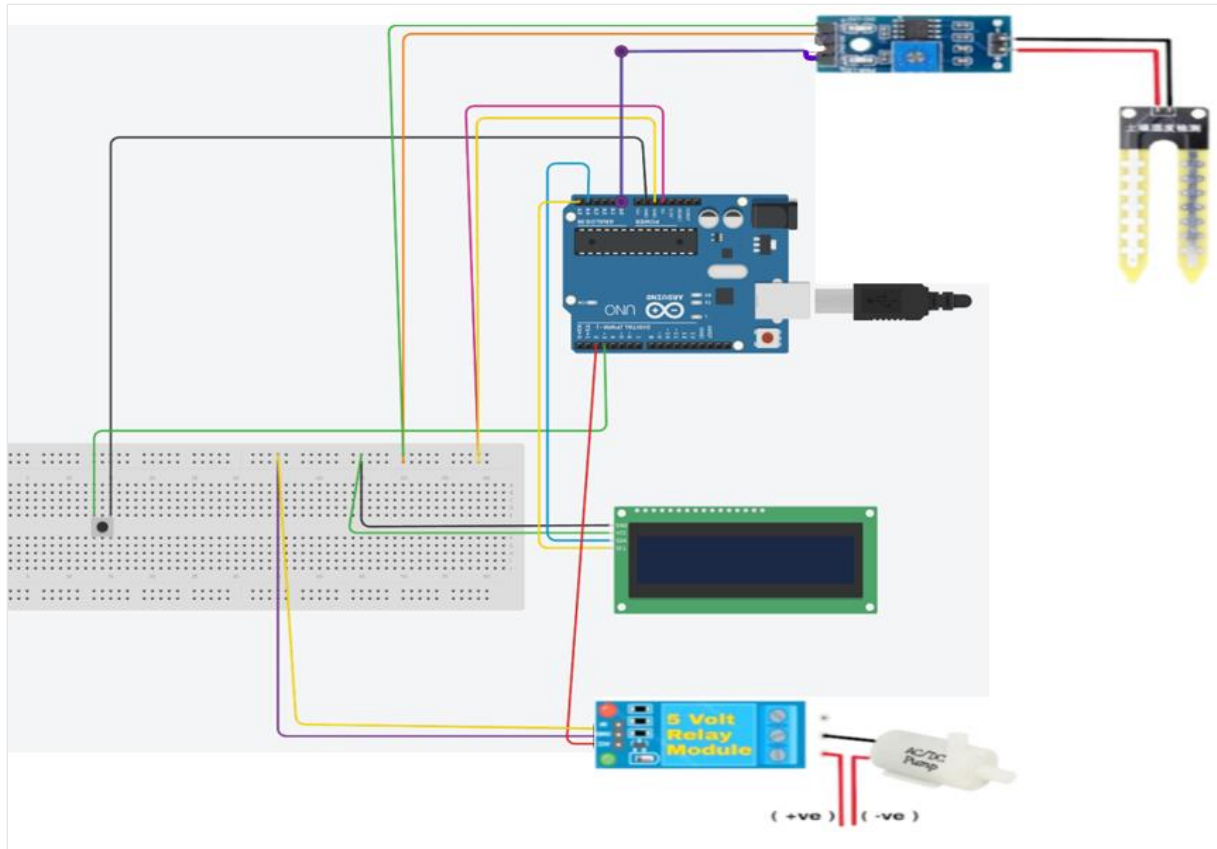


Figure 3. System schematic representation

3. The Arduino Code

3.1. How the Code Works

The device employs a moisture sensor to track the amount of moisture in the soil and an optional button to manually operate a water pump. When the soil is suitably moist, the pump automatically turns off, if it is too dry, it turns on. The system uses an LCD to display the soil moisture level and the pump's condition, and the button enables manual toggling of the pump's state.

Initialization:

A 16x2 LCD display is initialized using the LiquidCrystal_I2C library. Two pins are defined: one for controlling a water pump (pumpPin), and another for a push button (buttonPin). The pump is initially set to "OFF" (LOW), and the button state is tracked for debouncing.

Setup:

The LCD display is initialized and shows a welcome message: "IRRIGATION SYSTEM IS ON" for 3 seconds. The pump pin is set as an output, and the button pin is configured as an input with an internal pull-up resistor.

Main Loop:

The moisture level is continuously read from an analog moisture sensor (connected to pin A0). The state of the push button is checked, and debouncing is applied to avoid multiple button presses being

registered too quickly. If the button is pressed, the pump state toggles (ON/OFF), and the LCD updates with the current pump status. The pump's behavior is also controlled automatically based on the moisture sensor's readings, which states that, If the moisture value is low (greater than 700), the pump is turned ON, else, if the moisture value is high (less than 500), the pump is turned OFF. The moisture level is displayed on the LCD as "HIGH", "MID", or "LOW".

Display Updates:

The LCD also would show the current status of the pump (ON/OFF) and moisture levels ("HIGH", "MID", "LOW").

The code is represented as below:

```
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Pin definitions
const int pumpPin = 2; // Pin controlling the water pump
const int buttonPin = 3; // Pin connected to the push button
bool pumpState = LOW; // Initial pump state (OFF)
bool lastButtonState = HIGH; // To store the last button state
unsigned long lastDebounceTime = 0; // Debounce timer
unsigned long debounceDelay = 200; // Debounce delay (in milliseconds)

void setup() {
    Serial.begin(9600);
    lcd.init();
    lcd.backlight();
    lcd.clear();

    pinMode(pumpPin, OUTPUT);
    pinMode(buttonPin, INPUT_PULLUP); // Configure the button pin with internal arduino pull-up resistor
    digitalWrite(pumpPin, HIGH); // Set initial state to pump OFF

    delay(1000);

    lcd.setCursor(0, 0);
    lcd.print("IRRIGATION");
    lcd.setCursor(0, 1);
    lcd.print("SYSTEM IS ON");
    delay(3000);
    lcd.clear();
}
```

```

void loop() {
  int moistureValue = analogRead(A0); // Read the moisture sensor value
  Serial.println(moistureValue);

  // Read the button state
  int buttonState = digitalRead(buttonPin);

  // Check if the button state has changed and apply debouncing
  if (buttonState != lastButtonState) {
    lastDebounceTime = millis(); // Reset the debounce timer
  }

  if ((millis() - lastDebounceTime) > debounceDelay) {
    // If the button state has changed (pressed and released)
    if (buttonState == LOW) {
      pumpState = !pumpState; // Toggle pump state
      digitalWrite(pumpPin, pumpState); // Update the pump

      // Display pump status on the LCD
      lcd.clear();
      if (pumpState == HIGH) {
        lcd.setCursor(0, 0);
        lcd.print("Manual Contol ");
      } else {
        lcd.setCursor(0, 0);
        lcd.print("Manual Control");
      }
      delay(700); // Wait for the pump to stabilize (prevents multiple toggles)
    }
  }
}

```

```

// Update lastButtonState
lastButtonState = buttonState;

// Automatic control based on moisture level
if (moistureValue > 700 && pumpState == LOW) {
    digitalWrite(pumpPin, LOW); // Pump ON if moisture is low
    lcd.setCursor(0, 0);
    lcd.print("Water Pump is ON ");
} else if (moistureValue <= 500 && pumpState == LOW) {
    digitalWrite(pumpPin, HIGH); // Pump OFF if moisture is sufficient
    lcd.setCursor(0, 0);
    lcd.print("Water Pump is OFF");
}

// Display moisture level
if (moistureValue < 500) {
    lcd.setCursor(0, 1);
    lcd.print("Moisture : HIGH");
} else if (moistureValue >= 500 && moistureValue < 700) {
    lcd.setCursor(0, 1);
    lcd.print("Moisture : MID ");
} else {
    lcd.setCursor(0, 1);
    lcd.print("Moisture : LOW ");
}
}

```

4. Applications

The system can be implemented for Indoor gardening and houseplants, can also be applied in a small-scale setups. The process involved in performing the task can serve as educational project to demonstrate automation.

5. Future Development

To improve or expand the project for more functionality, efficiency, and scalability, the system can be upgraded by making a multi-Plant management, where by the system is expanded to support multiple plants by adding more soil moisture sensors and water pumps, using a multiplexer or additional pins to connect multiple sensors and control multiple relays. This would help manage an entire garden or multiple potted plants with one system.

I could implement IoT (Blynk console) in order to control the system remotely via a mobile phone.

6. Conclusions

- The system successfully automated plant watering by detecting soil moisture levels and activating a water pump as needed.
- The Integration of the Arduino Uno microcontroller with the soil moisture sensor, water pump, and relay worked as intended.

- Real-time soil moisture information and pump status were displayed on the LCD screen, providing valuable feedback.
- Watering was conducted only when necessary, reducing water waste.
- Energy consumption was minimized by operating the pump only when the soil was dry

Educational Value:

- The project demonstrated principles of sensor integration, and microcontroller programming.
- It provided practical experience in building and debugging an automated system.

Key Achievements:

- A fully functional prototype was built that can be scaled or customized for multiple plants or advanced features.
- The system proved effective in reducing manual effort for plant care, making it suitable for busy users or those frequently away from home.

Lessons Learned:

- The soil moisture sensor's readings and calibration is essential to ensure reliable operation.
- Combining hardware like sensors, relays, and pumps with software highlighted the importance of careful wiring and robust code to avoid malfunctions.
- Operating the system on battery power could improve portability and sustainability.

The Smart plant watering system successfully demonstrates how technology can simplify and optimize plant care. By merging automation, and practical design, the project provided a hands-on learning experience while creating a viable and effective solution. Future advances may expand its capabilities, making it more versatile and sustainable.